

Assignment 4

Name: Esraa Ashraf

ID: 224866

Part 1: Advanced Theoretical Assignment

Question 1: Concatenation and Sign Extension

Given:

wire signed [3:0] a = 4'sb1010;

assign result = {2{a}} >>> 2;

1. {2{a}} produces 8'b10101010. A concatenation is unsigned, so the arithmetic right shift treats it as unsigned.
2. Shifting 8'b10101010 right by 2 gives 8'b00101010. Therefore result = 8'b00101010.
3. The correct 8-bit sign extension is:
assign result = {{4{a[3]}}, a};
For a = 1010, this gives 8'b11111010.

Question 2: Logical vs. Bitwise Masking

1. mask_enable_logical = 4'b0001. Both vectors are non-zero, so the logical AND (&&) evaluates to 1.
2. mask_enable_bitwise = 4'b0000. The bitwise AND compares each bit: 0100 & 0010 = 0000.
3. Logical operators treat a whole multi-bit vector as a Boolean condition. They do not perform lane-by-lane masking. Bitwise operators preserve the individual bits, which is what is normally required when masking a multi-bit bus. For a single overall condition, a reduction operator such as |vector or &vector can be used.

Question 3: Operator Precedence and X-Propagation

For data = 4'b10X1 and ref = 4'b1001:

1. ^data is X because the reduction XOR contains an unknown bit.
2. data === ref) is 0 because case equality compares X as an actual value and the vectors are not identical.
3. (^data == (data === ref)) becomes X == 0, which evaluates to X.
4. The ternary condition is therefore X. Since the two possible results are 1'b1 and 1'b0, flag becomes X.
flag = 1'bx.

For Y: A + B = 4'b1111. The shift gives 4'b11110, which is greater than C = 4'b1100. Therefore the true branch is selected: A ^ B = 4'b1111. So Y = 4'b1111.

For Result: ^C = 0, |B = 1, ~&A = 1, and A[2:1] = 01. The concatenation is 7'b0001101 and is zero-extended to 8 bits, giving Result = 8'b00001101.

The identity operator (===) is mainly a four-state simulation construct for detecting X/Z values. Hardware does not have a separate physical X/Z comparison operation. A synthesizable comparison such as data == ref is implemented as ordinary bit comparisons combined with logic, typically producing an equality comparator.

Question 4: Conditional Operator vs. if-else

With a = 1, b = 0, and sel = X:

out_ternary = X. The conditional operator examines both possible values because sel is unknown, and the two values differ.

out_if = 0. In an if statement, X does not satisfy the true condition, so the else branch is executed.

If b is changed to 1 while a is also 1 and sel remains X:

out_ternary = 1 and out_if = 1.

Part 2: 8-Bit Multi-Function ALU

The alu_8bit module uses the required case statement inside an always block and implements all control codes from 00000 through 10110. The output is 16 bits as specified. The complete RTL is included in alu_8bit.v.

The testbench drives every control operation and includes edge cases such as 0xFF, 0x01, and negative arithmetic shift input.

Part 3: Data Scrambler

The Data_Scrambler module is fully combinational. It implements pass-through, key-based XOR scrambling, signed arithmetic right shift, nibble swapping, sleep-mode override, even-parity detection, and zero detection.

For arithmetic scaling, data_in is declared signed so >>> preserves its sign bit.

The testbench checks all four modes, a negative value during arithmetic shifting, zero/non-zero cases, and sleep mode.

Files Included

alu_8bit.v

tb_alu_8bit.v

Data_Scrambler.v

tb_Data_Scrambler.v